

EVETest Framework

(aka Eden 2.0)

Technical Design Document

Date: 24.10.2025

Author: Milan Lenco (milan@zedada.com)

Overview

EVETest (or simply just “evetest” with lowercase) is a next-generation testing framework for EVE-OS, designed to replace Eden and address its limitations. The framework enables comprehensive integration testing using virtualization, supports complex network scenarios, edge-node clustering, and provides a simplified developer experience.

Design Goals

- Simple configuration and minimal CLI commands
- Tests written using standard Go testing framework, with minimal boilerplate
- Support for multiple concurrent test instances on the same host
- Support for multiple EVE instances (for eve-k cluster tests)
- Flexible deployment: single-container for local development or distributed for CI/CD
- Programmable network configurations for comprehensive networking test coverage
- Multiple hypervisor support (libvirt, Proxmox, public cloud providers)
- Seamless integration with HTTP and docker proxies/caches
- Framework and tests co-located in EVE repository
- Practical GitHub CI/CD integration with per-test result reporting and rerunnable individual tests (e.g., compatible with [dorny/test-reporter](#)).

For a high-level proposal explaining why we need to evolve eden, see:

 [Proposal for Eden Evolution](#)

Architecture

EVETest consists of three primary components that work together to provide a complete testing environment. The framework's architecture is designed to be flexible, supporting both simple

all-in-one deployments for local development and sophisticated distributed deployments for CI/CD workflows.

High-Level Components

1. EVETest Container

The EVETest container is the heart of the testing system. It serves as the execution environment where your tests actually run. Inside this container, you'll find the Go test binary executing your test code, the Adam controller (EVE's open-source controller implementation), and Redis which Adam uses for state storage (we may get rid of this, see [MinimalistAdam](#)). The container also includes the EVETest CLI for interactive debugging and management.

What makes the EVETest container particularly flexible is that it can optionally include an embedded broker for all-in-one deployments. When running in this mode, everything you need to test EVE is contained within a single container, making it perfect for getting started, learning the framework, or working on your laptop during development.

The container is built with developer productivity in mind. It mounts your test directory from the EVE repository, giving you the ability to modify tests and immediately re-run them without rebuilding containers and EVE disk images. It also manages artifact collection, automatically gathering logs, console output and collect-info when tests fail.

Why run tests inside a container?

Running tests inside a container ensures a consistent, isolated, and reproducible environment. All dependencies with the correct versions are encapsulated within the container, eliminating the need to install or configure anything on the host system. This isolation also prevents tests from interfering with the host environment and guarantees safe, reliable cleanup afterward.

We've seen some practical issues with Eden that highlight these benefits. For example, test failures were sometimes caused by port collisions with other GitHub services running inside CI runners. In another case, Eden's cleanup process mistakenly removed personal, uncommitted files from a developer's repository. Running tests in an isolated container protects both the host system and developers' data from such unintended side effects.

The containerized approach also solves several networking and privilege-related challenges. To avoid running Eden under `sudo`, we previously relied on QEMU's user-space SLIRP networking, which proved very slow – especially for app image downloads. With the container, we can safely grant `NET_ADMIN` privileges, allowing the use of efficient `virtio/vhost` TAP interfaces and Linux bridges for EVE connectivity.

Currently, the only external dependency we rely on is QEMU itself. In the past, several issues arose when users attempted to run tests with incompatible QEMU versions that lacked support for certain configuration options. Encapsulating QEMU inside the container ensures that all

users and CI systems run with a known, verified version, avoiding version mismatch problems entirely.

Note that most of the container's contents are shared and reused across tests, so rebuilding the container is fast and benefits from cached layers.

2. Broker

The broker is the resource management and hypervisor abstraction layer. Its primary responsibility is to create and manage EVE virtual machines and the SDN (Software Defined Network) VM on behalf of test instances. Think of the broker as the component that translates high-level test requirements ("I need an EVE device with 4GB RAM and 2 network interfaces") into actual virtual machines running on a hypervisor.

The broker accomplishes this through a clean abstraction called the device provider interface. Different provider implementations allow the broker to work with different virtualization platforms – libvirt or direct qemu invocations for local development, Proxmox for CI/CD testing (if we decide to use it on Kratos), or even public cloud providers (similar to the recently removed GCP workflow that we had in the eden repository). Each provider implementation handles the specifics of that platform's API while presenting a consistent interface to the broker.

One of the broker's key features is its ability to manage multiple test instances concurrently. It tracks which resources are allocated to which test, ensures that tests don't interfere with each other, and handles cleanup when tests complete. The broker also implements intelligent caching of EVE container images – when multiple tests need the same EVE version, the image is built once and reused, significantly speeding up test execution.

In distributed deployments, the broker takes on an additional networking role: it acts as a tunnel proxy, forwarding network traffic between the EVETest container (which might be running on a small GitHub runner) and the SDN VM (running on a powerful server). This tunneling happens transparently, allowing tests to interact with EVE devices as if they were running locally, regardless of where the VMs actually reside.

3. SDN (Software Defined Network)

The SDN component is what enables EVETest to test complex networking scenarios. It's a specialized VM that models physical network infrastructure using software. When your test needs to verify that EVE works correctly behind a proxy, on a VLAN, or with port-based authentication, the SDN provides that environment.

The SDN runs as a separate VM, created by the broker for each test instance. EVE devices connect to ports on the SDN, and the SDN provides their network connectivity – whether that's direct Internet access, access through a proxy, DHCP configuration, or more complex scenarios involving multiple network segments.

What makes the SDN powerful is its programmability. Tests can define a declarative network model that specifies exactly how the network should be configured: which ports are on which VLANs, where DHCP servers should run, what firewall rules are in place, DNS config to advertise, and so on. The SDN applies this configuration and provides a realistic network environment for testing. It can even report on connectivity status, allowing tests to skip scenarios that require resources (like IPv6 Internet connectivity) that aren't currently available.

The SDN is based on the [eden-sdn](#) work but has been enhanced with a gRPC API for consistency with other EVETest components and includes support for establishing IP tunnels when operating in distributed mode.

Why is the SDN implemented as a VM?

Implementing the SDN as a VM, rather than running it directly on the host or inside a container, provides several key advantages:

- **Isolation and safety:** Running network emulation directly on the host would require administrative privileges and could interfere with host networking, posing security and stability risks.
- **Flexibility for complex topologies:** Implementing SDN as a container would make it harder, and in some cases impossible to represent multiple networks, routers, and subnets since we would be limited to only one network namespace. Inside a VM we can create and interconnect multiple network namespaces.
- **Simplified Layer 2 connectivity:** EVETest needs to create L2 links between EVE interfaces and the SDN. This is naturally supported when both EVE and SDN are VMs (a capability provided by any hypervisor), but becomes complex when mixing VMs with containers.

Note that the SDN VM is quite small (built using linuxkit) and does not require much resources.

Deployment Modes

EVETest's architecture is designed for flexibility, supporting a wide range of use cases through two distinct deployment modes.

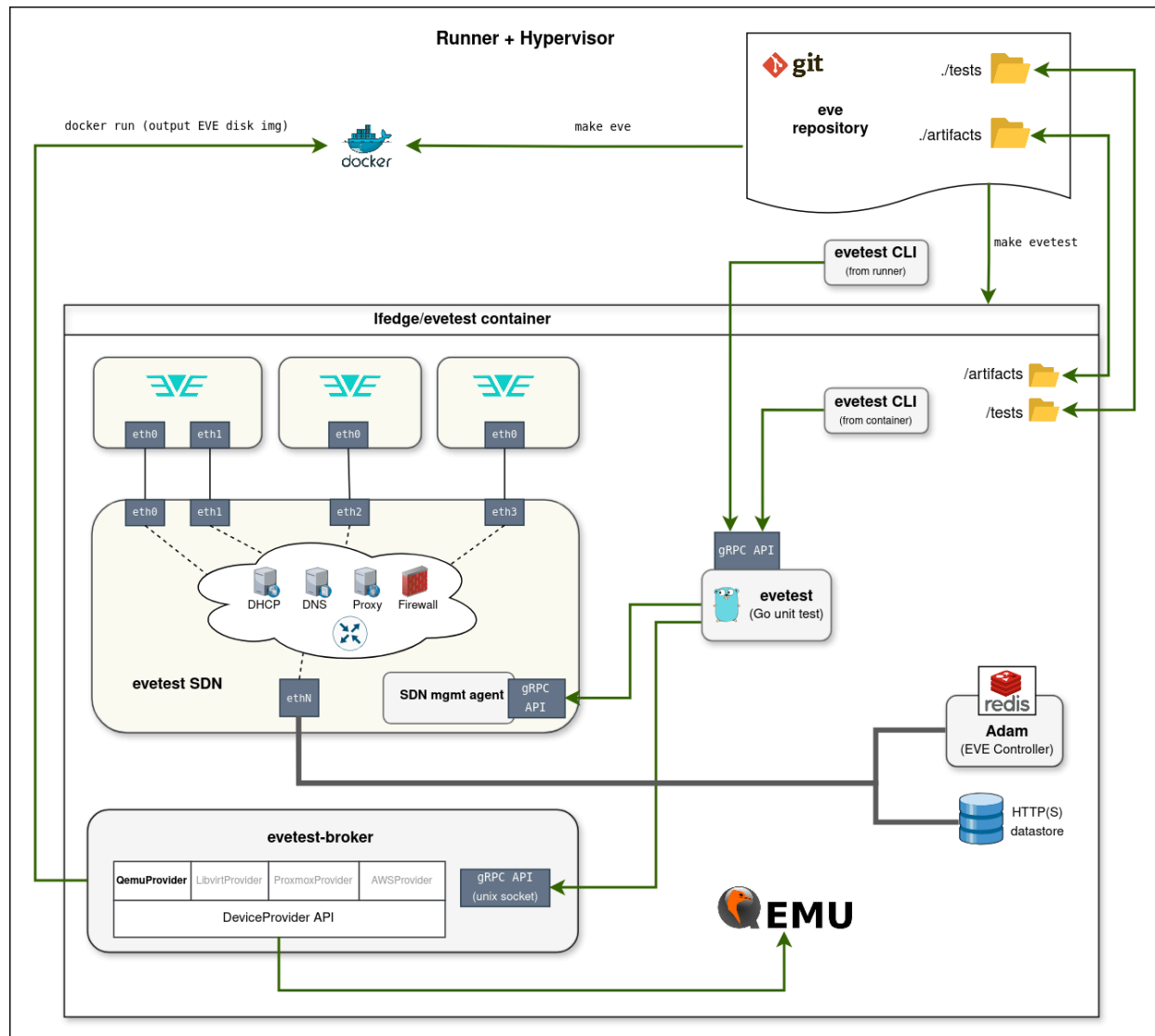
All-in-One Mode: Simplicity for Development

In all-in-one mode, everything runs within a single EVETest container. This is the mode you'll likely use when developing EVE (especially eve-kvm) on your laptop, when introducing new team members to EVETest, or when you want the simplest possible setup.

When you start a test in this mode, the EVETest container launches and immediately starts all necessary services: Adam, Redis, and an embedded broker. This embedded broker uses QEMU to create virtual machines directly inside the container. The small but not zero overhead

caused by running VMs inside the container is acceptable for EVE development and learning purposes.

In this mode, the broker and EVETest communicate via a Unix domain socket since they're in the same container. The SDN VM and EVE VMs are created as QEMU virtual machines within the container's namespace. Because everything shares the same network namespace, there's no need for complex tunneling – the EVETest container can directly communicate with the SDN and EVE devices.



This mode is perfect for:

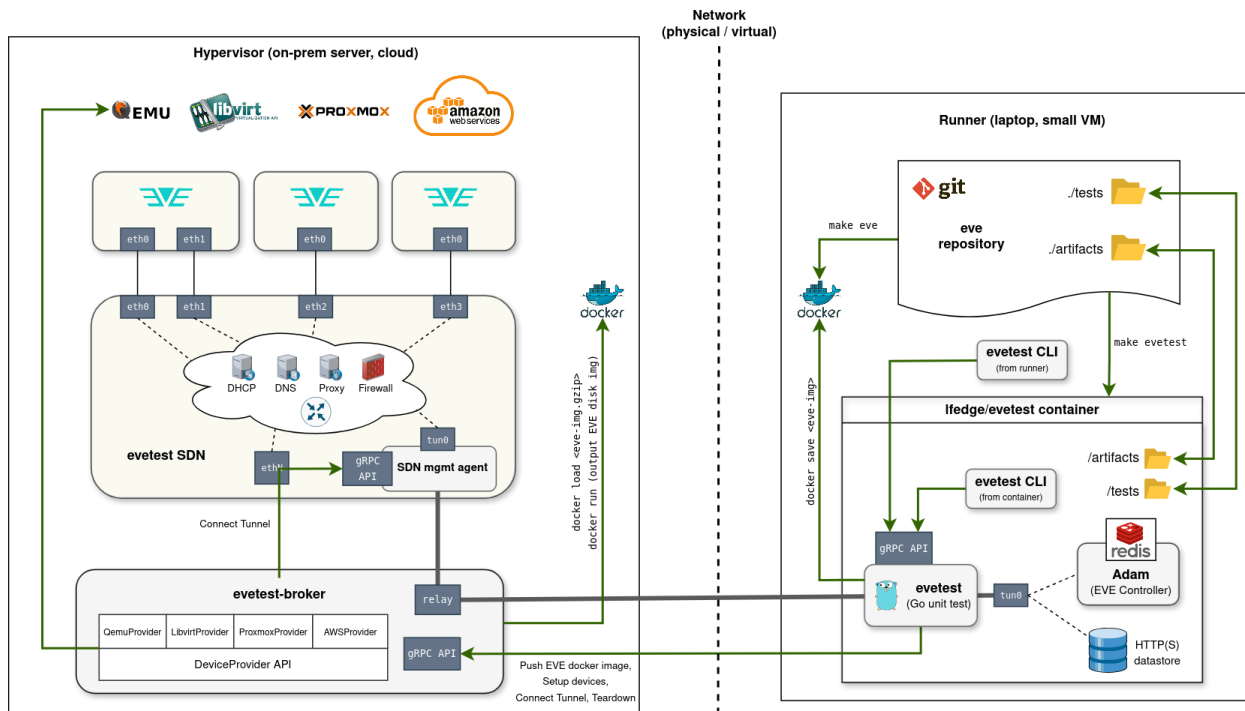
- Local development on laptops (especially when only one EVE instance running is needed)
- Learning EVETest and experimenting with tests
- Quick iteration during test development

- Scenarios where you don't have access to a remote hypervisor
- Introducing EVE to open-source community

The all-in-one mode requires nested virtualization support in your CPU and hypervisor, but most modern development machines provide this. You would run it with a single command:

`make evetest NAME=mytestname`, and everything just works.

Distributed Mode: Power and Scalability for CI/CD



Distributed mode is where EVETest shows its true power. In this deployment model, the EVETest container runs on one machine (often a small, resource-constrained GitHub runner or your laptop) while the actual EVE VMs run on a completely different, more powerful server with a proper hypervisor.

This separation addresses the issue of excessive nested virtualization that we currently face in the Eden setup, which often leads to test flakiness. With the new setup, EVE VMs run directly on the hypervisor, even when evetests are invoked by CI/CD on virtualized runners.

Here's how the pieces fit together in distributed mode: The EVETest container starts on the runner and immediately establishes a gRPC connection to the broker (IP+port passed by config vars) running on the hypervisor server. Once your test calls `evetest.Setup()` to request EVE devices (and a particular network model), the EVETest container sends this request to the

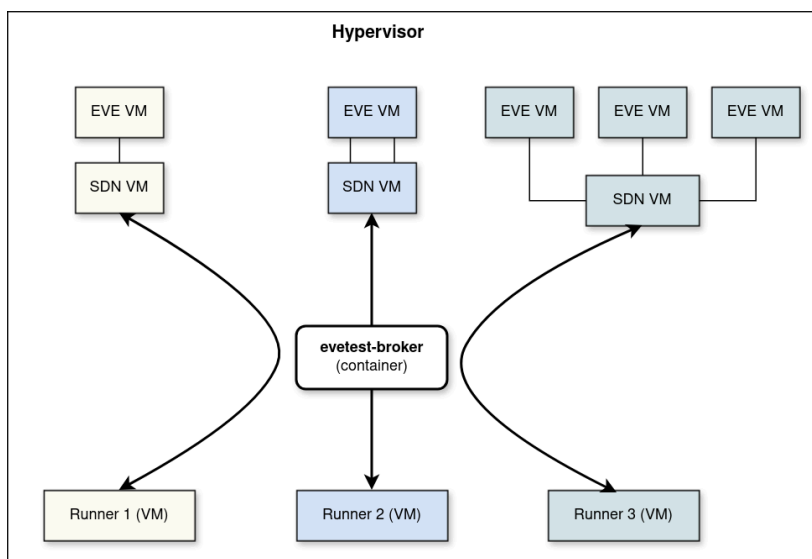
broker over gRPC. The broker then uses its device provider implementation (let's say libvirt) to create the requested VMs on the hypervisor server.

But there's a networking challenge: your test code running in the EVETest container needs to communicate with EVE devices that are running on a different server, possibly behind a firewall and/or NAT, possibly on a completely different network. Likewise, EVE devices must be able to access the Adam controller, running inside the EVETest container. The plan is to solve this with IP tunneling. When the broker creates the SDN VM, it also establishes a tunnel – essentially a TCP connection over which raw IP packets are forwarded. The broker sits in the middle of this tunnel and acts as a data-plane proxy, receiving packets from the SDN and forwarding them to the EVETest container, and vice versa.

From your test's perspective, this is completely transparent. When you call methods that depend on SSH connectivity to an EVE device or to an application running inside, the framework handles all the complexity of routing traffic through the tunnel. The EVETest container creates a tunnel interface (using its NET_ADMIN capability), configures appropriate routes, and your test code simply communicates with EVE devices using IP addresses as if everything were local.

This distributed architecture enables some powerful scenarios:

- Run tests on your laptop while EVE VMs execute on the Kratos server (connection is over the office network or VPN)
- GitHub workflows with tiny runners invoking tests that launch more powerful EVE VMs on a dedicated (Kratos or other) server
- Multiple developers or CI jobs running tests concurrently, all sharing the same hypervisor infrastructure:



The broker manages all the complexity of resource allocation, ensuring that concurrent tests don't interfere with each other and that resources are properly cleaned up when tests complete (or abruptly disconnect and do not reconnect within a time limit).

Component Interactions

Understanding how these components communicate helps clarify the architecture. In all-in-one mode, the interaction pattern is straightforward: the test code calls EVETest API methods, which communicate with Adam over REST, with the embedded broker over a Unix socket, and with the SDN via direct network connections.

In distributed mode, the pattern becomes more sophisticated but remains clean from the test developer's perspective. The test code still calls the same EVETest API methods. When these methods need to interact with Adam or datastores, they communicate directly since those services run in the same container. When they need to interact with the broker (to create VMs, power devices on/off, access console output), they use gRPC over the network. When they need to interact with EVE devices or the SDN, the traffic flows through the IP tunnel, with the broker acting as a relay proxy.

The EVETest CLI adds another layer of flexibility. The CLI can run inside the EVETest container (where it's pre-installed) or on your host machine (if you decide to install it on your system). In either case, it communicates with the EVETest container via gRPC. When you run commands like `evetest eve ssh`, the CLI asks the EVETest container to establish a TCP proxy, which it then uses to forward your SSH connection through the tunnel to the EVE device. This proxy-within-proxy architecture might sound complex, but it enables a seamless developer experience where you can SSH into EVE devices, copy files, view logs, and interact with tests regardless of where anything is actually running.

Core Components Deep Dive

EVETest Container

The EVETest container is packaged as a standard Docker container image based on a Golang image with additional utilities for debugging and troubleshooting (tcpdump, strace, bash, etc.). When you run `make evetest`, this container is built (or pulled from cache) and started with specific configurations.

The container exposes several mount points that integrate it with your development workflow. The `/tests` directory is mounted from your EVE repository's test directory, allowing you to edit tests on your host and immediately re-run them inside the container. The `/artifacts` directory is optionally mounted to collect test outputs, logs, and diagnostic information. The container also needs access to the Docker socket (mounted as a volume) so it can access EVE container images.

Inside the container, the default command launches a shell script that ultimately executes `go test` with the appropriate flags. Your test code runs as a standard Go test, calling EVETest API methods as needed. The framework handles starting Adam and Redis, establishing broker

connection, and managing the test lifecycle. When tests complete or fail, the framework automatically collects artifacts before EVE devices are torn down and the container exits.

The container runs with elevated privileges (NET_ADMIN capability or privileged mode) because it needs to create tunnel interfaces and manipulate routing tables in distributed mode. In all-in-one mode, these privileges also allow it to run QEMU and create the necessary network bridges for VM connectivity.

Broker

The broker serves as the bridge between test requirements and actual virtual machine infrastructure. It exposes a gRPC API that EVETest containers use to request resources, manage device lifecycles, and access device consoles.

When a test calls `Setup()` with requirements for EVE devices, the EVETest container translates this into a series of broker gRPC calls. First, it may need to build EVE images by calling the broker's `BuildImage` method, which builds an EVE live or installer image with the appropriate configuration (certificates, controller address, bootstrap settings). If the broker doesn't have the required EVE container image, it responds with an error, prompting the EVETest container to push the container image data (obtained from the local Docker registry via the mounted docker socket).

Once images are ready, the test calls `SetupDevices`, providing specifications for each EVE device (CPU count, memory, disk size, network interfaces) and the SDN. The broker uses its device provider implementation to create these VMs on the underlying hypervisor. The device provider interface abstracts the specifics of each platform – libvirt implementations use libvirt APIs, Proxmox implementations use Proxmox REST APIs, and so on.

The broker maintains state about which resources belong to which client (identified by a client ID). This enables concurrent testing – multiple EVETest instances can connect to the same broker, each managing their own set of VMs without interference. When a client disconnects abruptly (test completes), the broker marks resources for cleanup but implements a retention period for EVE container images, allowing subsequent tests to reuse them.

In distributed mode, the broker takes on the additional role of tunnel proxy. When an EVETest container requests a tunnel connection via `ConnectTunnelToSDN`, the broker establishes a separate gRPC stream for data plane traffic. It then launches a goroutine that continuously reads IP packets from the SDN VM and forwards them to the EVETest container, and vice versa. This forwarding happens at wire speed and is transparent to both the SDN and the test code.

The broker also provides device management capabilities. Methods like `PowerOnDevice`, `PowerOffDevice`, and `RebootDevice` allow tests (and the CLI) to control device power

states. The `GetDeviceConsoleOutput` and `ConnectConsoleToDevice` methods provide access to device consoles, which is invaluable for debugging boot or networking issues.

Link to proposed Broker gRPC API:

<https://github.com/milan-zededa/eve/blob/evetest/evetest/grpcapi/proto/broker.proto>

Device Provider Interface

The device provider interface is the abstraction that allows the broker to support multiple hypervisors and cloud platforms. Each provider implements a standard Go interface that defines methods for creating VMs, managing power states, accessing consoles, and querying available resources.

The interface design focuses on VM lifecycle operations. Methods like `SetupDevice` accept a specification structure that describes the desired VM (CPU, memory, disk, network interfaces) and return a VM identifier. The provider implementation handles all the platform-specific details—for libvirt, this means generating domain XML and calling libvirt APIs; for Proxmox, this means making REST API calls with the appropriate parameters.

Power management methods (`PowerOnDevice`, `PowerOffDevice`, `RebootDevice`) abstract the control operations that tests frequently need. The provider ensures these operations complete correctly and handles any platform-specific quirks (like waiting for shutdown confirmation before declaring success).

Console access is particularly important for debugging. The `GetDeviceConsoleOutput` method retrieves the console buffer, allowing you to see boot messages and kernel output. The `AttachToDeviceConsole` method provides bidirectional console access, enabling interactive debugging sessions. Not all providers can support these features—it depends on the underlying platform's capabilities.

The initial implementation focuses on libvirt (for local development) and QEMU (for embedded broker mode). Future implementations might add Proxmox support (if we decide to use it for Kratos) and potentially public cloud providers (AWS, Azure).

Link to proposed Device Provider Interface:

<https://github.com/milan-zededa/eve/blob/evetest/evetest/broker/provider/provider.go>

SDN (Software Defined Network)

The SDN component is a specialized VM running software that can model complex network topologies and configurations. It's built on the foundation of eden-sdn but enhanced for integration with EVETest.

When the broker creates an SDN VM, it starts with a standard SDN container image that includes all the necessary networking tools and services. The SDN exposes a gRPC API that EVETest containers use to configure the declarative network model and query connectivity status.

A network model defines the entire network topology: physical interfaces (represented as virtual interfaces in the VM), LAGs, VLANs, DHCP server configurations, routing tables, firewall rules, proxy servers, and any authentication mechanisms like 802.1X (to-be-implemented). When a test applies a network model via the SDN API, the SDN configures all these components using standard Linux networking tools and services.

EVE devices connect to the SDN just as they would to physical network switches. Each EVE VM has network interfaces that connect to specific SDN ports. The SDN then handles their traffic according to the configured model—applying VLAN tags, routing packets, serving DHCP leases, proxying HTTP requests, or enforcing authentication.

The SDN also provides connectivity checking capabilities. Before a test proceeds, it can query whether the SDN has IPv4 or IPv6 Internet connectivity. Tests can use this information to skip scenarios that require resources that aren't available (for example, skipping an IPv6 test if the test environment doesn't have IPv6 connectivity).

In distributed mode, the SDN includes special logic for tunnel management. When the broker requests a tunnel, the SDN creates a tunnel interface, assigns IP addresses and routes, and begins forwarding packets over the gRPC connection to the broker. From the SDN's perspective, the tunnel networks are just another part of the "outside" network topology.

Network model proto definition can be found here:

<https://github.com/milan-zededa/eve/blob/evetest/evetest/grpcapi/proto/sdn.proto>

EVETest CLI

The EVETest CLI is a command-line tool that developers use to interact with running test instances. It provides a rich set of commands for inspecting test state, managing EVE devices, viewing logs, and debugging issues.

The CLI communicates with EVETest containers via gRPC, connecting to a port that the container exposes. When you run the CLI from your host machine, it connects to the (selected or the only running) EVETest container port (which is mapped to a host port). When you run the CLI from inside the container (by executing it in a shell within the container), it connects to localhost.

The CLI is organized into several command categories. Test control commands (**continue**, **restart**, **exit**, **status**) manage the test execution flow. These are particularly useful when

you've paused a test at a checkpoint or on failure—you can inspect the state, make changes if needed, and then continue.

EVE device management commands provide access to device configuration, information messages, and metrics. Commands like `evetest eve config` show you what configuration has been pushed to a device, while `evetest eve info` displays the device information that EVE publishes to the controller. The `-f` flag on many commands enables follow mode, continuously displaying updates as they arrive.

Log access commands are essential for debugging. You can view all EVE logs, console output, application-specific logs, or flow logs. The CLI automatically handles the complexity of retrieving these logs regardless of where the EVE device is actually running.

Remote access commands (`ssh`, `scp`, `console`) deserve special mention because they handle complex networking transparently. When you run `evetest eve ssh`, the CLI makes a gRPC call to the EVETest container, which establishes a TCP proxy. The proxy endpoint is returned to the CLI, which then uses it to make the actual SSH connection. The EVETest container forwards traffic from this proxy through the tunnel (in distributed mode) or directly to the SDN (in all-in-one mode), which then forwards it to the EVE device. This multi-hop proxy architecture means you can SSH into EVE devices with a simple command, regardless of the deployment topology.

Cluster commands (`cluster info`, `cluster metrics`) provide visibility into EVE Kubernetes cluster state when testing eve-k. These commands aggregate information across multiple EVE devices to show the cluster topology and health.

Link to sketched EVETest CLI:

<https://github.com/milan-zededa/eve/blob/evetest/evetest/TODO.md#evetest-binary-the-cli>

Test Development Experience

Writing Tests

Tests in EVETest are standard Go tests that use the Go testing framework. This means you write tests exactly as you would write any Go test, using familiar patterns and tools. The EVETest framework provides a simple API that you call from your test functions to set up EVE devices, configure networks, and interact with running systems.

A typical test begins with `evetest.Init(t)`, which initializes the framework. You should immediately defer `evetest.Close()` to ensure proper cleanup regardless of how the test exits.

Optionally, you can call `evetest.DefineTestParameters(params...)` to specify parameters and make the test parameterized, avoiding code duplication. For example, you can

test onboarding with a static IP using either a bootstrap configuration or an `override.json` file.

Next, invoke `evetest.Setup(requirements, networkModel)`, where `requirements` defines what the test needs — such as the number of EVE devices and their resource specifications — and `networkModel` describes the SDN configuration to apply.

The Setup call is where most of the magic happens. Behind the scenes, EVETest evaluates whether it can reuse existing devices from a previous test (if you're running a test suite) or whether it needs to create new ones. It builds EVE images if necessary, creates VMs through the broker, establishes tunnels, applies network configurations, waits for devices to boot and onboard, and validates that everything is ready. When Setup returns, you have fully operational EVE devices ready for testing.

You interact with EVE devices through the EdgeDevice object returned by `evetest.GetEdgeDevice(name)`. This object provides methods that mirror the operations you'd perform manually: deploying applications, changing configurations, reading logs, running SSH commands, rebooting devices. The framework handles all the complexity of actually executing these operations, whether that means making Adam API calls, establishing SSH connections through proxies, or waiting for configuration changes to get applied.

Tests can include checkpoints using `evetest.Checkpoint(name)`. When a test reaches a checkpoint with a name that matches the `EVETEST_PAUSE_ON_CHECKPOINT` environment variable, execution pauses. This is invaluable for debugging—you can set a checkpoint, run the test, and when it pauses at that point, use the CLI to inspect logs, SSH into devices, and understand what's happening before allowing the test to continue.

The beauty of this approach is its simplicity. You're not learning a custom test language or wrestling with complex frameworks. If you know Go and the Go testing package, you can write EVETest tests. The framework provides just enough abstraction to make common operations easy while still giving you full control when you need it.

Here is an example of a simple test checking that onboarding with static IP works correctly:

https://github.com/milan-zededa/eve/blob/evetest/tests/networking/bootstrap_test.go#L92-L179

Test Suites

Test suites in EVETest are just Go tests that run multiple subtests. They are defined using `evetest.RunTestSuite` and behind the scenes use `t.Run()` to define Go subtests.

What makes test suites powerful in EVETest is resource reuse. When multiple tests in a suite have similar requirements, the framework can often reuse devices rather than tearing everything down and recreating it. For example, if your test suite runs 10 networking tests that all need a basic two-interface EVE setup, the first test creates the devices and the subsequent tests reuse

them (possibly with reboots to clear state – cleaning method can be specified in the test requirements).

The framework provides suite-level failure limit through the `EVETEST_SUITE_MAX_FAILURES` environment variable. You can specify how many tests are allowed to fail before the suite aborts. Setting this to -1 means the suite runs all tests regardless of failures, useful for CI/CD where you want to see all results. Setting it to 1 (the default) means the suite aborts after the first failure, useful for development where you want to fix issues immediately.

Here are a couple of example network test suites:

https://github.com/milan-zededa/eve/blob/evetest/tests/networking/testsuite_test.go

Configuration

Configuration in EVETest is deliberately simple and uses environment variables exclusively. This design decision avoids the complexity of configuration files while making it easy to see exactly what settings are in effect.

Most tests only need to set two or three variables: `EVETEST_NAME` to specify which test to run, `EVETEST_BROKER_ADDRESS` to point to a broker (or leave unset for the default all-in-one deployment mode), and optionally `EVETEST_COLLECT_ARTIFACTS` to specify where to store outputs.

Other variables control debugging behavior (`EVETEST_PAUSE_ON_CHECKPOINT`, `EVETEST_PAUSE_ON_FAILURE`), logging (`EVETEST_LOG_LEVEL`), and version selection (`EVETEST_EVE_VERSION`, `EVETEST_ADAM_VERSION`). The framework provides sensible defaults for everything, so you only need to set variables when you want non-default behavior.

For developers, common configurations can be put in shell scripts or make targets. For CI/CD, GitHub workflow files specify the variables directly. This approach makes configurations explicit, visible, and easy to understand.

Workflows

Developer Workflow

The developer workflow is designed to be as frictionless as possible. You start by checking out the EVE repository and the branch you want to work on. If you've made changes to EVE itself, you run `make eve` to build the EVE container image that will be used for testing.

To run a test, you simply execute `make evetest NAME=yourtest`. The Makefile handles building the EVETest container (cached, so this is fast after the first time), starting it with the

appropriate mounts and environment variables, and executing your test. The output appears in your terminal as the test runs.

When the test fails, you can debug it by setting `EVETEST_PAUSE_ON_FAILURE=true`. This prevents the test environment from tearing down, allowing you to inspect the state. You can then either enter the EVETest container with `docker exec -it <container> bash` and use the CLI from inside, or install the CLI on your host with `make install-evetest-cli` and use it from outside (`evetest <cmd> args..`).

The CLI gives you full access to the test environment. You can view logs, check device status, SSH into EVE devices, run commands, inspect configurations—anything you need to understand what's happening. When you're done investigating, you run `evetest continue` to let the test proceed (or just exit if it is paused on failure) or `evetest exit` to clean up and stop.

If you're iterating on test code itself, you can modify the test file (since `/tests` is mounted from your host) and run `evetest restart`. This rebuilds and re-runs the test while reusing the existing EVE VMs, which is much faster than recreating everything from scratch.

This workflow supports both all-in-one mode (everything on your laptop) and distributed mode (using a remote hypervisor). The experience is the same either way—the framework handles the complexity transparently.

CI/CD Workflow

The GitHub CI/CD workflow leverages EVETest's distributed architecture and container isolation. Workflow files are clean and simple because most of the complexity is hidden inside the EVETest framework.

A typical workflow runs on a self-hosted runner (which can be a small VM) and starts by checking out the EVE repository. It loads the EVE container image from a cache populated by the build workflow. Then it sets a few environment variables—the test (suite) name, broker address, and artifact path—and runs `make evetest`.

The EVETest container starts on the runner, connects to the remote broker (typically running on Kratos or another powerful server), and executes the test. The broker creates EVE VMs on the hypervisor server, and the test runs, with all traffic flowing through the established tunnel. When the test completes, artifacts are written to the specified path.

The workflow then publishes test results (output from `go test -json`) and uploads artifacts for failed tests. Because EVETest uses standard Go testing JSON output, you can use existing GitHub Actions like [dorny/test-reporter](https://github.com/dorny/test-reporter) to display results in the GitHub UI.

Multiple workflows can run concurrently. Each gets its own EVETest container instance, connects to the broker with a unique client ID, and manages its own isolated set of VMs. The broker ensures they don't interfere with each other.

This architecture means your runners can be small and numerous (cheap to run, easy to scale) while still running tests against properly-resourced EVE VMs on capable hardware. It's the best of both worlds: cloud-native CI/CD with realistic testing conditions.

Benefits and Trade-offs

Advantages Over EDEN

EVETest addresses all the major pain points identified in EDEN:

Simplified User Experience: Starting a test requires one command instead of a sequence of carefully-ordered CLI invocations. Configuration is straightforward environment variables instead of multiple complicated config files (in eden called “contexts”).

Better Code Organization: Tests are regular Go tests in the EVE repository, eliminating the complexity of coordinating changes across two repositories and dealing with release dependencies. This allows publishing PRs containing both a feature/bug-fix implementation and a corresponding test.

Reduced Nesting: Distributed mode eliminates the excessive virtualization layers that cause flakiness in EDEN. EVE VMs run directly on proper hypervisors with full hardware support.

Network Testing: Complete SDN integration enables testing scenarios that are impossible in current EDEN (VLANs, proxies, authentication, complex routing, IPv6).

Concurrent Testing: Container isolation and broker resource management allow multiple tests to run simultaneously from the same host without conflicts.

Multi-Device Support: The architecture naturally supports testing multiple EVE devices concurrently, enabling eve-k cluster testing. But we also need to enhance Adam to support cluster configuration.

Hypervisor Flexibility: The device provider abstraction means you're not locked into QEMU (and a particular version) – use libvirt locally, Proxmox for Kratos, or even cloud providers.

Trade-offs and Considerations

The distributed mode architecture introduces network complexity that must be managed carefully. The tunnel proxy adds latency (although I didn't notice it in my preliminary POC) and requires the broker to maintain connection state. However, this complexity is encapsulated in the framework and invisible to test developers.

Running `evetest` in distributed mode is more complex to set up than EDEN. It requires a server running the broker with hypervisor access and properly configured user accounts and access rights. However, once configured, this setup can support many concurrent tests from multiple developers and CI jobs, making it an efficient shared resource.

In distributed mode, the broker acts as a single point of failure. If it crashes or restarts, some VMs may remain running and become “zombies”. We need to implement resource cleanup carefully.

Migration Path

The transition from EDEN to EVETest will be gradual and practical. Both frameworks will coexist during a transition period, allowing teams to adopt EVETest at their own pace while maintaining existing EDEN tests.

Priority tests—those that run frequently or test critical functionality—should be ported to EVETest first. The porting process is generally straightforward since EVETest's Go test API is more expressive than EDEN's escript language. Complex EDEN workflows often become simpler EVETest tests.

New features should be tested exclusively with EVETest. This ensures the framework gets real-world usage and feedback while gradually building up the EVETest test suite. Over time, as EVETest's test coverage grows and confidence in the framework increases, EDEN tests can be deprecated.

The ultimate goal is for EVETest to completely replace EDEN, but this happens when it's ready—when it has achieved feature parity, when the community is comfortable with it, and when the migration has proven the framework's reliability.

Implementation Roadmap

Implementation proceeds in phases, each delivering working functionality:

Phase 1 (MVP) focuses on the core framework in all-in-one mode. This means getting the basic EVETest API working, the embedded broker with QEMU provider, single-device testing, artifact collection, and essential CLI commands. This phase proves the concept and provides a usable tool for simple tests.

Phase 2 implements distributed mode, building the standalone broker, libvirt provider, tunnel proxying, and remote access proxying. This phase enables CI/CD deployment and resource-intensive testing.

Phase 3 adds advanced features like multi-device support for cluster testing, the Proxmox provider for Kratos (maybe), and enhanced resource management (more robust cleanup, monitoring, etc.). This phase makes EVETest fully production-ready.